

---

# Curvature-Guided Parameter Initialization for Multi-Task Learning

---

Linxiao Cao<sup>1\*</sup> Zhipeng Zhou<sup>2\*</sup> Xutao Huang<sup>1</sup> Min Zhou<sup>3</sup> Menglin Yang<sup>1†</sup>

<sup>1</sup>The Hong Kong University of Science and Technology (Guangzhou)

<sup>2</sup>Nanyang Technological University

<sup>3</sup>Huawei Technologies Ltd.

{lcao950, xhuang225}@connect.hkust-gz.edu.cn

zzpustcml@gmail.com zhoumin27@huawei.com menglinyang@hkust-gz.edu.cn

## Abstract

Multi-task learning (MTL) aims to jointly optimize multiple related tasks to obtain shared representations within a single model. Recent research has focused on developing MTL algorithms to alter optimization dynamics through task re-weighting or gradient manipulation. However, in this paper, we first empirically identify that current MTL approaches are highly sensitive to the model initialization, complicating empirical evaluation and limiting practical reliability, which has been overlooked but appears to be critical. To better understand this phenomenon, we provide a local theoretical analysis that motivates the connection between early-stage MTL optimization behavior and curvature around initialization. Motivated by this analysis, we propose a Curvature-guided Parameter Initialization (CPI) approach tailored for MTL. Specifically, CPI performs a short warm-up phase to probe local curvature around the initial parameters and heuristically biases the initialization toward empirically favorable curvature profiles for subsequent multi-task optimization. The proposed approach is optimizer-agnostic, requires no modification to existing MTL algorithms, and can be seamlessly integrated as a plug-in prior to standard training. Extensive experiments on standard MTL benchmarks show that CPI improves the aggregate multi-task trade-off while remaining compatible with mainstream MTL methods.

## 1 Introduction

Multi-task learning (MTL) seeks to improve generalization by jointly learning multiple related tasks through shared representations, which allow related tasks to share statistical strength [Zhang and Yang, 2021, Cao et al., 2026, Zhou et al., 2024]. By exploiting task relatedness, MTL has been successfully applied to a wide range of domains, including computer vision [Crawshaw, 2020, Zhou et al., 2025a,b] and natural language processing [Chen et al., 2024], offering improved data efficiency and inductive bias over single-task learning (STL).

A large body of recent work [Ban and Ji, 2024, Liu et al., 2021a] has focused on mitigating optimization challenges arising from task competition. Representative approaches include dynamic loss reweighting [Chen et al., 2018, Liu et al., 2023], gradient-based conflict resolution [Yu et al., 2020, Sener and Koltun, 2018, Liu et al., 2021a, Ban and Ji, 2024, Zhou et al., 2025c], and explicit modeling of task relationships [Liu et al., 2019]. These methods primarily operate during training to balance task objectives and have led to consistent improvements in average performance across standard MTL benchmarks.

---

\*Equal contribution.

†Corresponding author.

Despite recent progress, we observe that the optimization outcomes of MTL methods can vary substantially across different random seeds, even under identical architectures and hyperparameter settings (see Figure 2 (a) - (b) and Section 3.1). Further analysis indicates that this variability is closely related to sensitivity to model initialization: when other sources of randomness are controlled, small changes in the initial parameters can still lead to noticeably different empirical outcomes (see Figure 3 and Section 3.1). Together, these observations point to initialization sensitivity as an important contributor to the observed seed-wise variability in MTL.

Moreover, the effects of these perturbations (shown in Figure 3 ) are not unidirectional: while some perturbations degrade performance, others lead to improvements. This observation suggests that, even within a local neighborhood of a given standard initialization, alternative parameter configurations may exist that yield more favorable multi-task optimization outcomes. In contrast, directly applying STL initialization strategies to the MTL setting fails to consistently improve performance (Figure 2 (c) - (d)), which naturally raises the question of how initialization can be more deliberately chosen to facilitate effective multi-task optimization.

Prior work on STL has noted that local curvature is closely related to optimization behavior near initialization [Fort and Scherlis, 2018, Vyso-gorets et al., 2024]. Inspired by this line of work, we explore curvature as a lens for analyzing initialization in MTL and develop theoretical insights into this connection (Section 3.3). Building on these insights, we propose a curvature-guided parameter initialization (CPI) approach for MTL (Figure 1). CPI exploits local curvature information in the vicinity of standard initializations to bias the model toward nearby regions with empirically favorable local geometry, while remaining fully compatible with existing MTL optimization methods.

We summarize our contributions as follows:

- To the best of our knowledge, we provide the first systematic empirical analysis demonstrating that MTL optimization is highly sensitive to initialization, highlighting the importance of initialization and the need for principled initialization strategies tailored to MTL.
- Guided by motivational theoretical analysis indicating that positive and bounded curvature is favorable for stable and well-conditioned early-stage multi-task optimization, we propose CPI for MTL, which leverages finite-difference directional curvature information to bias initial parameters toward empirically favorable local geometry.
- Through extensive experiments on standard MTL benchmarks, we empirically demonstrate that CPI improves the aggregate multi-task trade-off across diverse datasets, while remaining seamlessly compatible with a wide range of existing MTL algorithms.

## 2 Related Work

### 2.1 Multi-Task Learning

Existing MTL approaches can be broadly categorized into architecture-based and optimization-based methods. While architecture-based methods design shared or partially shared structures to facilitate knowledge transfer across tasks [Heuer et al., 2021, Yang and Hospedales, 2016, Gao et al., 2019, He et al., 2021, Liu et al., 2022, Chen et al., 2023, Fan et al., 2022], optimization-based methods directly model task interactions during training and are more closely related to our work. Representative approaches include multi-objective formulations such as MGDA [Sener and Koltun, 2018], gradient-space conflict mitigation methods such as PCGrad [Yu et al., 2020], CAGrad [Liu et al., 2021a], GO4Align [Shen et al., 2024], and ConsMTL [Qin et al., 2025b], as well as game-theoretic and fairness-aware methods such as Nash-MTL [Navon et al., 2022], FAMO [Liu et al., 2023], and FairGrad [Ban and Ji, 2024]. Other recent studies further analyze or optimize MTL from

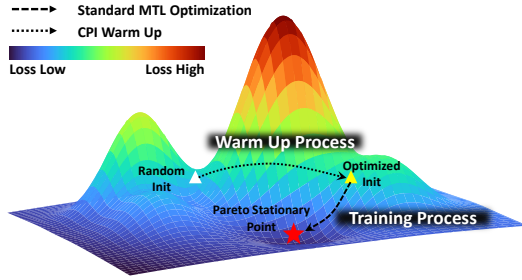


Figure 1: Overview of CPI. CPI performs a curvature-guided warm-up from a standard initialization, steering the shared parameters toward nearby regions with more favorable directional-curvature profiles before multi-task training.

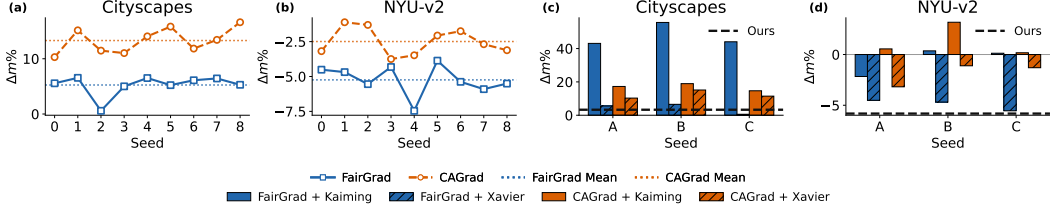


Figure 2: Panels (a)–(b) show seed-wise  $\Delta m\%$  of CAGrad and FairGrad. Panels (c)–(d) compare Kaiming and Xavier initialization under three fixed seeds. Lower  $\Delta m\%$  is better.

representation and training-dynamics perspectives, including Rep-MTL [Wang et al., 2025] and NTKMTL [Qin et al., 2025a].

Despite their methodological diversity, existing MTL approaches largely focus on training dynamics after initialization. Both architecture-based and optimization-based methods implicitly assume that random initialization provides a sufficiently neutral starting point. As a result, the role of initialization in shaping multi-task optimization trajectories has received limited attention. This gap motivates our work on initialization-aware MTL.

## 2.2 Initialization and Optimization Geometry

Initialization plays a critical role in deep learning by shaping early training dynamics and determining which regions of the loss landscape are explored. Classical schemes such as Xavier [Glorot and Bengio, 2010] and Kaiming [He et al., 2015] stabilize signal propagation by preserving activation and gradient variance, while data-dependent methods such as LSUV [Mishkin and Matas, 2015] further adjust parameter scales using empirical activation statistics. More recent studies have linked optimization stability and generalization to geometric properties of the loss landscape, including curvature [Chaudhari et al., 2019], sharpness [Foret et al., 2020], and Hessian spectra [Sagun et al., 2016]. From this perspective, initialization serves not only as a means of numerical stabilization, but also as a geometric prior that biases subsequent optimization toward different regions of the landscape.

However, most existing initialization strategies are developed in the context of single-task learning (STL). Unlike STL, MTL involves heterogeneous task gradients and curvature, resulting in more complex optimization geometry and task interactions. Existing initialization methods do not explicitly account for these interactions, while geometry-aware techniques are typically applied during training rather than to shape the initial parameters. This motivates our approach, which leverages task-wise curvature to construct a more favorable initialization for MTL.

## 3 Motivation

In this section, we first present empirical evidence demonstrating the sensitivity of MTL to initialization. We then show that initialization strategies directly transferred from STL are insufficient, while more favorable alternatives exist. Finally, we provide theoretical insights that motivate our approach.

### 3.1 Initialization-Induced Instability

To characterize the extent of seed-wise variability in modern MTL, we evaluate two representative gradient-based MTL methods, FairGrad [Ban and Ji, 2024] and CAGrad [Liu et al., 2021a], on Cityscapes [Cordts et al., 2016] and NYU-v2 [Silberman et al., 2012]. For each method–dataset pair, we repeat training multiple times while varying only the random seed, keeping the architecture, optimization algorithm, and all hyperparameters fixed.

As shown in Figure 2 (a) – (b), both methods exhibit substantial fluctuations in final performance across different random seeds on both datasets. Notably, the observed variability is not confined to a small number of outlier runs, but instead manifests consistently across seeds, indicating a pronounced sensitivity of these MTL methods to randomness in training.

To disentangle the effect of random seeds from that of initialization, we fix the random seed and explicitly probe the sensitivity of multi-task optimization to small perturbations of the initial shared parameters. Concretely, starting from a standard initialization determined by a given seed, we introduce multiple small noise perturbations to the initial parameters and train the model under otherwise identical settings. Implementation details are provided in Appendix A.1.2. Figure 3 illustrates the resulting behavior on Cityscapes.

Despite originating from the same random seed, these minor perturbations lead to markedly different final performance outcomes. This observation suggests that, even when the source of randomness is fixed, the optimization behavior of multi-task models can vary substantially in response to fine-grained changes in the initial parameter configuration. Consequently, performance instability in MTL cannot be attributed solely to coarse differences across random seeds, but is also strongly influenced by sensitivity to the initial parameter region.

### 3.2 MTL-Tailored Initialization is Needed

A natural question arising from the above observations is whether initialization strategies originally developed for STL can mitigate the instability observed in MTL. To investigate this, we apply commonly used STL initialization schemes, including Kaiming [He et al., 2015] and Xavier [Glorot and Bengio, 2010], to the multi-task setting. Throughout these experiments, we keep the model architecture, optimization algorithm, and training protocol fixed, varying only the initialization scheme.

As shown in Figure 2 (c) - (d), under identical training conditions, different initialization schemes result in clearly different multi-task performance profiles across random seeds, corroborating the initialization-related variability observed earlier. Moreover, none of the examined STL-oriented initialization schemes yields stable and consistently strong performance across datasets and seeds, indicating that their effectiveness does not straightforwardly transfer to the multi-task setting.

While existing STL-oriented initialization schemes fail to provide improvements in the multi-task setting, Figure 3 reveals that the effects of small perturbations around a standard initialization are not unidirectional: some perturbations lead to degraded performance, whereas others yield clear improvements relative to the unperturbed. These observations suggest that more favorable starting points do exist locally, motivating the development of initialization strategies tailored to MTL.

### 3.3 Motivational Theoretical Insights

Having observed that favorable MTL initialization points may exist near standard initializations, we next ask how such points can be identified in a principled way. Motivated by prior work on STL initialization [Fort and Scherlis, 2018, Vysogorets et al., 2024] and by our empirical evidence of initialization sensitivity in MTL (Sec. 3.1), we provide a local motivational analysis showing that positive and bounded task-wise curvature at initialization can support well-conditioned early-stage optimization. This insight guides us to use local curvature information when adjusting the initialization before standard multi-task training.

Consider an MTL problem with task losses:

$$f_t : \mathbb{R}^D \rightarrow \mathbb{R}, \quad t = 1, \dots, T, \quad (1)$$

and denote  $g_t(\theta) = \nabla f_t(\theta)$ ,  $H_t(\theta) = \nabla^2 f_t(\theta)$ . For  $\lambda \in \Delta_T$ , the MTL’s combined objective is:

$$F_\lambda(\theta) = \sum_t \lambda_t f_t(\theta), \quad g_\lambda(\theta) := \nabla F_\lambda(\theta) = \sum_{t=1}^T \lambda_t g_t(\theta) \quad (2)$$

and the positive but moderate curvature at initialization  $\theta_0$  can be defined as follows:

$$m_0 I \leq H_t(\theta_0) \leq L_0 I, \quad 0 < m_0 < L_0 < \infty. \quad (3)$$

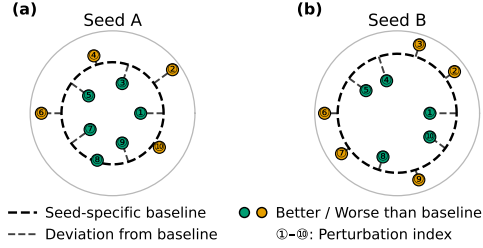


Figure 3: Each panel shows perturbation-induced performance variations with respect to a seed-specific baseline (dashed circle). Colored markers denote individual perturbation instances, where points closer to the center indicate better overall performance.

We analyze initialization points  $\theta_0$  where all task Hessians are locally positive and bounded, and show that this condition implies a stable early optimization phase with guaranteed descent and linear convergence, with an explicit rate governed by the curvature pair  $(m_0, L_0)$ .

**Assumption 3.1** (Hessian Lipschitz). For all  $i$ ,  $\|H_t(u) - H_t(v)\|_{op} \leq \rho_t \|u - v\|$ .

Let  $\rho := \max_t \rho_t$ . Define the (explicit) safe radius and local curvature constants  $r := \frac{m_0}{2\rho}$ ,  $m_r := m_0 - \rho r = \frac{m_0}{2}$ ,  $L_r := L_0 + \rho r = L_0 + \frac{m_0}{2}$ .

**Theorem 3.2** (Safe Descent and Convergence). Consider MTL gradient descent:  $\theta_{i+1} = \theta_i - \eta g_\lambda(\theta_i)$ .

Let  $\eta \leq 1/L_r$ . Suppose also that the initialization satisfies  $\|g_\lambda\| \leq \frac{m_r r}{2} = \frac{m_0^2}{8\rho}$ . Then:

(1) All iterates remain in the curvature-controlled ball:

$$\|\theta_i - \theta_0\| \leq r, \forall i \geq 0. \quad (4)$$

(2) For all  $i \geq 0$ ,

$$F_\lambda(\theta_{i+1}) \leq F_\lambda(\theta_i) - \frac{\eta}{2} \|g_\lambda(\theta_i)\|^2. \quad (5)$$

(3) Let  $\theta_\lambda^*$  be the unique minimizer of  $F_\lambda$  inside  $\mathcal{B}(\theta_0, r)$ . Then

$$\|\theta_i - \theta_\lambda^*\| \leq (1 - \eta m_r)^i \|\theta_0 - \theta_\lambda^*\|. \quad (6)$$

The permissible stable step size and rate depend explicitly on the curvature pair at initialization:

$$\eta \leq \frac{1}{L_0 + m_0/2}, \quad (1 - \eta m_r) \leq 1 - \frac{m_0/2}{L_0 + m_0/2}. \quad (7)$$

Thus, moderate curvature (smaller  $L_0$ ) allows larger stable steps and faster contraction, while positive curvature ( $m_0 > 0$ ) yields a strictly stronger guarantee, which in general cannot be obtained when curvature at initialization is negative or uncontrolled.

This analysis is intended as a local sufficient-condition motivation rather than a global guarantee for deep non-convex MTL. Such a scope is inherent to tractable analyses of neural-network initialization, and the result is used here to identify the curvature profile that CPI aims to encourage.

## 4 Methodology

### 4.1 Problem Setup

We consider an MTL setting with  $T$  tasks trained jointly using a shared backbone. Let  $\theta \in \mathbb{R}^D$  denote the model parameters, and let  $\mathcal{L}_t(\theta)$  be the loss of task  $t \in \{1, \dots, T\}$ . During training, an MTL optimizer aggregates the task gradients  $\nabla_\theta \mathcal{L}_t(\theta)$  to update the shared parameters.

We study the sensitivity of multi-task optimization to the initial parameters  $\theta_0$ , focusing on how initialization affects the subsequent optimization trajectory under a fixed training procedure.

To mitigate this sensitivity, we propose a curvature-guided initialization procedure that adjusts the starting point through a short warm-up phase before standard training, yielding  $\theta_{\text{init}} = \theta_0 + \Delta\theta$ . An overview of the proposed procedure is shown in Figure 1.

### 4.2 Overview

We propose CPI for MTL that operates as a short warm-up stage prior to standard optimization. The objective of CPI is to bias the initialization toward nearby parameter regions with empirically favorable directional-curvature profiles, motivated by the positive and bounded curvature condition in Section 3.3.

At a **conceptual** level, CPI can be viewed as implicitly optimizing a warm-up objective of the form

$$\mathcal{L}_{\text{warm}}(\theta) = \mathcal{L}_{\text{cur}}(\theta) + \mathcal{L}_{\text{stab}}(\theta), \quad (8)$$

---

#### Algorithm 1 CPI for MTL

---

**Require:** Initial shared parameters  $\theta_0$ , task losses  $\{\mathcal{L}_t\}_{t=1}^T$

**Ensure:** Initialized parameters  $\theta_{\text{init}}$

```

1: Initialize  $\theta \leftarrow \theta_0$ 
2: for initialization step  $s = 1, \dots, S$  do
3:   for each task  $t = 1, \dots, T$  do
4:     Compute  $g_t = \nabla_\theta \mathcal{L}_t(\theta)$ 
5:     Estimate the directional curvature proxy  $\kappa_t(\theta)$ 
6:     Construct  $u_t^{\text{curv}}$  from  $\kappa_t(\theta)$ 
7:     Project  $u_t^{\text{curv}}$  w.r.t.  $g_t$  to obtain  $\tilde{u}_t^{\text{curv}}$  using Eq. (15)
8:   end for
9:    $\Delta\theta \leftarrow \sum_{t=1}^T \tilde{u}_t^{\text{curv}}$ 
10:   $\theta \leftarrow \theta + \eta \Delta\theta$ 
11: end for
12: return  $\theta_{\text{init}} \leftarrow \theta$ 

```

---

where  $\mathcal{L}_{\text{cur}}$  encourages favorable local directional-curvature profiles of task-specific losses, and  $\mathcal{L}_{\text{stab}}$  constrains first-order task-loss variations so that the warm-up remains within a local neighborhood of the initial parameters. Importantly, CPI does not explicitly minimize  $\mathcal{L}_{\text{warm}}$  by backpropagation, since directly differentiating curvature-based objectives would require higher-order derivatives and is computationally prohibitive for high-dimensional networks.

Instead, CPI constructs update directions using finite-difference directional curvature probes. As summarized in Algorithm 1, CPI first estimates task-wise directional curvature at the current parameters using stochastic probes. It then constructs curvature-guided update directions that heuristically penalize negative directional curvature and overly large positive directional curvature. Finally, CPI projects these updates to remove components aligned with task gradients, thereby limiting first-order loss changes and encouraging the warm-up to explore nearby parameter regions with similar loss but more favorable local geometry.

### 4.3 Estimating Task-wise Curvature

CPI estimates task-wise local curvature through stochastic directional probes. Instead of explicitly forming the Hessian matrix or computing its spectrum, both of which are computationally prohibitive for modern neural networks, CPI approximates directional second-order information using finite differences of gradients. For each task  $t$ , we first compute the gradient  $\nabla_{\theta}\mathcal{L}_t(\theta)$  at the current parameters. Given a randomly sampled unit direction  $v \in \mathbb{R}^D$ , we approximate the directional curvature of task  $t$  via finite differences of gradients,

$$v^{\top}\nabla^2\mathcal{L}_t(\theta)v \approx v^{\top}\frac{\nabla_{\theta}\mathcal{L}_t(\theta + \sigma v) - \nabla_{\theta}\mathcal{L}_t(\theta)}{\sigma}, \quad (9)$$

where  $\sigma > 0$  is a small step size. Equation (9) provides a single-sample estimate of curvature along direction  $v$ . To reduce variance, we sample multiple probe directions and average the corresponding estimates, yielding a stochastic directional curvature proxy, denoted by  $\kappa_t(\theta)$ .

### 4.4 Constructing Curvature-Guided Updates

In this section, we construct a Hessian-free curvature-guided parameter update based on the finite-difference directional-curvature surrogate motivated by Section 3.3, thereby avoiding explicit Hessian computation and direct optimization of Hessian-based objectives. To instantiate this Hessian-free update, we first estimate local task-wise curvature characteristics around the current parameters using symmetric finite differences. Specifically, for each task  $t$ , we probe the curvature proxy along a randomly sampled unit direction  $h \in \mathbb{R}^D$  by evaluating  $\kappa_t(\theta)$  at perturbed parameters  $\theta + \sigma h$  and  $\theta - \sigma h$ . We then define

$$K_t = \frac{1}{2}(\kappa_t(\theta + \sigma h) + \kappa_t(\theta - \sigma h)), \quad (10)$$

which serves as a local estimate of the magnitude and sign of task-wise curvature around  $\theta$ , and

$$s_t = \frac{\kappa_t(\theta + \sigma h) - \kappa_t(\theta - \sigma h)}{2\sigma}, \quad (11)$$

which provides a finite-difference approximation of the directional variation of the curvature proxy along  $h$ .

As suggested by the local analysis in Section 3.3, positive and bounded task-wise curvature provides a desirable local geometry for early-stage MTL optimization. Motivated by this observation, we design curvature-guided updates that explicitly correct negative curvature while preventing excessively large positive curvature. For each task  $t$ , the Hessian-free curvature-guided update is defined as

$$u_t^{\text{curv}} = -\text{sign}(K_t) \cdot \text{sign}(s_t) \cdot \alpha_t(K_t) \cdot h, \quad (12)$$

where the signs of  $K_t$  and  $s_t$  determine whether the probe direction should be followed or reversed. Intuitively, when  $K_t < 0$ , the update favors directions that increase the estimated curvature toward zero; when  $K_t \geq 0$ , it suppresses directions that would further increase already positive curvature. Thus, the rule provides a Hessian-free heuristic for reducing unfavorable directional-curvature indicators, subject to probe and finite-difference approximation errors.

Since the magnitude of  $s_t$  is sensitive to the finite-difference step size  $\sigma$  and stochastic estimation noise, we set the update scale using a nonnegative curvature-dependent factor:

$$\alpha_t(K_t) = \begin{cases} \rho |K_t|, & K_t < 0, \\ (1 - \rho) |K_t|, & K_t \geq 0, \end{cases} \quad (13)$$

which balances the relative emphasis between correcting negative curvature and suppressing large positive curvature.

#### 4.5 Controlling First-Order Loss Effects

The curvature-guided update  $u_t^{\text{curv}}$  adjusts the initialization using local curvature information, but a parameter update also affects task losses. If strongly aligned with the task gradient, such an update may be dominated by first-order loss decrease, driving the parameters toward steep descent directions and sharp regions. Besides, as illustrated in Figure 3, better initialization points could exist within a local neighborhood of standard initializations. Motivated by these observations, we explicitly control first-order loss variations in order to explore nearby regions with similar loss but more favorable geometry.

A first-order Taylor expansion of the task loss,

$$\mathcal{L}_t(\theta + \Delta\theta) \approx \mathcal{L}_t(\theta) + \nabla_{\theta} \mathcal{L}_t(\theta)^{\top} \Delta\theta, \quad (14)$$

the leading-order change in  $\mathcal{L}_t$  is governed by the inner product between the update direction and the task gradient. We therefore control this term to suppress first-order effects.

Specifically, for each task  $t$ , we project  $u_t^{\text{curv}}$  onto the subspace orthogonal to the corresponding task gradient. The projected update is given by

$$\tilde{u}_t^{\text{curv}} = u_t^{\text{curv}} - \frac{\langle u_t^{\text{curv}}, \nabla_{\theta} L_t(\theta) \rangle}{\|\nabla_{\theta} L_t(\theta)\|_2^2 + \epsilon} \nabla_{\theta} L_t(\theta). \quad (15)$$

where  $\epsilon > 0$  is a small constant for numerical stability. This projection ensures that the task-wise update satisfies  $\nabla_{\theta} \mathcal{L}_t(\theta)^{\top} \tilde{u}_t^{\text{curv}} \approx 0$ , thereby suppressing the first-order loss variation at the task level. The final initialization update is obtained by aggregating the projected task-wise updates,  $\Delta\theta = \sum_{t=1}^T \tilde{u}_t^{\text{curv}}$ , and the resulting parameters are taken as the final initialization  $\theta_{\text{init}}$ .

## 5 Evaluation

### 5.1 Experimental Setup

**Tasks and Datasets.** We evaluate CPI on four standard MTL benchmarks covering **classification, regression, and dense prediction**. CelebA [Liu et al., 2015] is used as a 40-task binary attribute classification benchmark following [Liu et al., 2023]. QM9 [Ramakrishnan et al., 2014] is used for 11-task molecular property regression with the standard 110K/10K/10K split [Fey and Lenssen, 2019]. For dense prediction, we use NYU-v2 [Silberman et al., 2012] with semantic segmentation, depth estimation, and surface normal prediction, and Cityscapes [Cordts et al., 2016] with semantic segmentation and depth estimation. Both dense prediction benchmarks use a SegNet backbone with MTAN heads following [Liu et al., 2023].

**Evaluation Metrics.** Following standard MTL evaluation practice [Liu et al., 2023], we use the mean relative performance change,  $\Delta m\%$ , as the **primary** comparison criterion. Since MTL aims to improve the **overall trade-off** across heterogeneous tasks rather than dominate every individual metric,  $\Delta m\%$  provides a scale-normalized aggregate measure:  $\Delta m\% = \frac{1}{T} \sum_{t=1}^T (-1)^{\delta_t} \cdot \frac{M_{m,t} - M_{b,t}}{M_{b,t}} \times 100$ . Here,  $M_{m,t}$  and  $M_{b,t}$

Table 1: Results on *CelebA* (40-task) and *QM9* (11-task) datasets.

| Method                        | CelebA                  | QM9                     |
|-------------------------------|-------------------------|-------------------------|
|                               | $\Delta m\% \downarrow$ | $\Delta m\% \downarrow$ |
| LS                            | 4.15                    | 177.6                   |
| SI                            | 7.20                    | 77.8                    |
| RLW [Lin et al., 2021]        | 1.46                    | 203.8                   |
| DWA [Liu et al., 2019]        | 2.40                    | 175.3                   |
| UW [Kendall et al., 2018]     | 3.23                    | 108.0                   |
| MGDA [Sener and Koltun, 2018] | 14.85                   | 120.5                   |
| PCGrad [Yu et al., 2020]      | 3.17                    | 125.7                   |
| IMTL-G [Liu et al., 2021b]    | 0.84                    | 77.2                    |
| CAGrad [Liu et al., 2021a]    | 2.48                    | 112.8                   |
| NashMTL [Navon et al., 2022]  | 2.84                    | 62.0                    |
| FAMO [Liu et al., 2023]       | 1.21                    | 58.5                    |
| FairGrad [Ban and Ji, 2024]   | <b>0.37</b>             | 57.9                    |
| FairGrad-R                    | 1.09                    | 59.9                    |
| COST [Zhou et al., 2025b]     | 0.93                    | 58.3                    |
| <b>CPI (Ours)</b>             | <b>0.52</b>             | <b>55.2</b>             |

denote the multi-task and single-task performance on task  $t$ , and  $\delta_t$  encodes the metric direction. Lower  $\Delta m\%$  indicates a better overall MTL trade-off, with negative values meaning that MTL outperforms STL on average.

## 5.2 Baselines

We compare CPI with representative MTL baselines covering loss weighting, gradient manipulation, and fairness-aware optimization.

**Loss weighting methods** adjust task importance through static or adaptive weights, including LS, SI, DWA [Liu et al., 2019], UW [Kendall et al., 2018], and RLW [Lin et al., 2021]. **Gradient-based methods** modify task gradients to reduce interference, including MGDA [Sener and Koltun, 2018], PCGrad [Yu et al., 2020], CAGrad [Liu et al., 2021a], IMTL-G [Liu et al., 2021b], and COST [Zhou et al., 2025b]. **Fairness-aware methods** seek balanced task performance via equilibrium or constrained optimization objectives, including NashMTL [Navon et al., 2022], FAMO [Liu et al., 2023], and FairGrad [Ban and Ji, 2024]. For dense prediction benchmarks, we additionally include GradDrop [Chen et al., 2020], MoCo [Fernando et al., 2023], and F-CAGrad [Phan et al., 2025], which target structured gradient conflicts or generalization in MTL.

## 5.3 Experimental Results

We follow standard experimental protocols from prior MTL studies, with full training details provided in Appendix A.1.2. **Consistent with common practice in MTL, all results are averaged over three random seeds** to ensure a controlled and comparable evaluation budget across methods. Additional analyses, including **component ablations, curvature analysis, efficiency evaluation, hyperparameter sensitivity, error bars, and comparisons with alternative initialization baselines**, are provided in Appendices A.4–A.11.

For baselines, we use publicly available implementations under unified experimental settings. When reported numbers differ from our reproduced results, we use the reproduced results as the primary comparison.<sup>3</sup> In the tables, CPI is highlighted in gray, and the best results are shown in **bold**.

### 5.3.1 Main Results.

**Multi-task Classification and Regression.** On CelebA, CPI achieves a  $\Delta m\%$  of 0.52, showing competitive performance among existing MTL baselines and improving upon the reproduced FairGrad baseline. On QM9, CPI achieves the lowest  $\Delta m\%$  of 55.2, outperforming competing methods such as FAMO (58.5) and FairGrad-R (59.9). These results indicate that CPI improves the aggregate multi-task trade-off across both image-level attribute classification and heterogeneous molecular property regression benchmarks.

**Dense Prediction.** Table 2 reports the results on Cityscapes, which involves joint semantic segmentation and depth estimation. CPI achieves the lowest overall  $\Delta m\%$  of 2.74, outperforming all competing baselines. While a few methods obtain strong performance on individual tasks (e.g., F-CAGrad), CPI achieves a better overall trade-off across tasks, which highlights the effectiveness of CPI in balancing heterogeneous dense prediction objectives.

Table 2: Results on Cityscapes (2-task) dataset.

| Method            | Seg. $\uparrow$ |              | Depth $\downarrow$ |              | $\Delta m\% \downarrow$ |
|-------------------|-----------------|--------------|--------------------|--------------|-------------------------|
|                   | mIoU            | Pix. Acc.    | Abs.               | Rel.         |                         |
| STL               | 74.01           | 93.16        | 0.0125             | 27.77        | -                       |
| LS                | 75.18           | 93.49        | 0.0155             | 46.77        | 22.60                   |
| RLW               | 74.57           | 93.41        | 0.0158             | 47.79        | 24.37                   |
| DWA               | 75.24           | 93.52        | 0.0160             | 44.37        | 21.43                   |
| Uncertainty       | 72.02           | 92.85        | 0.0140             | 30.13        | 5.88                    |
| MGDA              | 68.84           | 91.54        | 0.0309             | 33.50        | 44.14                   |
| GradDrop          | 75.27           | 93.53        | 0.0157             | 47.54        | 23.67                   |
| PCGrad            | 75.13           | 93.48        | 0.0154             | 42.07        | 18.21                   |
| IMTL              | 75.33           | 93.49        | 0.0135             | 38.41        | 11.04                   |
| CAGrad            | 75.16           | 93.48        | 0.0141             | 37.60        | 11.58                   |
| NashMTL           | 75.41           | 93.66        | <b>0.0129</b>      | 35.02        | 6.82                    |
| FAMO              | 74.54           | 93.29        | 0.0145             | 32.59        | 8.13                    |
| FairGrad          | 75.72           | 93.68        | 0.0134             | 32.25        | 5.18                    |
| FairGrad-R        | 74.61           | 93.36        | 0.0131             | 31.47        | 4.23                    |
| F-CAGrad          | <b>76.02</b>    | <b>93.72</b> | 0.0134             | 34.64        | 7.25                    |
| COST              | 75.73           | 93.53        | 0.0133             | 31.53        | 4.30                    |
| <b>CPI (Ours)</b> | <b>73.76</b>    | <b>92.98</b> | <b>0.0133</b>      | <b>28.94</b> | <b>2.74</b>             |

Table 3 presents the results on NYU-v2, that jointly considers semantic segmentation, depth estimation, and surface normal prediction. CPI achieves the best overall  $\Delta m\%$  of  $-5.82$ , surpassing all baseline methods. Notably, while some approaches excel on specific metrics, CPI consistently improves trade-off performance across multiple task groups, including surface normal estimation.

<sup>3</sup>Methods marked with the suffix -R denote reproduced results under the same protocol.

Table 3: Results on *NYU-v2* (3-task) dataset.

| Method      | Segmentation $\uparrow$ |              | Depth $\downarrow$ |             | Surface Normal              |              |                             |              |              | $\Delta m\%$ $\downarrow$ |
|-------------|-------------------------|--------------|--------------------|-------------|-----------------------------|--------------|-----------------------------|--------------|--------------|---------------------------|
|             | mIoU                    | Pix. Acc.    | Abs. Err.          | Rel. Err.   | Angle Distance $\downarrow$ |              | Within $t^\circ$ $\uparrow$ |              |              |                           |
|             |                         |              |                    |             | Mean                        | Median       | 11.25                       | 22.5         | 30           |                           |
| STL         | 38.30                   | 63.76        | 0.68               | 0.28        | 25.01                       | 19.21        | 30.14                       | 57.20        | 69.15        | -                         |
| LS          | 39.29                   | 65.33        | 0.55               | 0.23        | 28.15                       | 23.96        | 22.09                       | 47.50        | 61.08        | 5.46                      |
| RLW         | 37.17                   | 63.77        | 0.58               | 0.24        | 28.27                       | 24.18        | 22.26                       | 47.05        | 60.62        | 7.67                      |
| DWA         | 39.11                   | 65.31        | 0.55               | 0.23        | 27.61                       | 23.18        | 24.17                       | 50.18        | 62.39        | 3.49                      |
| Uncertainty | 36.87                   | 63.17        | 0.54               | 0.23        | 27.04                       | 22.61        | 23.54                       | 49.05        | 63.65        | 4.01                      |
| MGDA        | 30.47                   | 59.90        | 0.61               | 0.26        | 24.88                       | 19.45        | 29.18                       | 56.88        | 69.36        | 1.47                      |
| GradDrop    | 39.39                   | 65.12        | 0.55               | 0.23        | 27.48                       | 22.96        | 23.38                       | 49.44        | 62.87        | 3.61                      |
| PCGrad      | 38.06                   | 64.64        | 0.56               | 0.23        | 27.41                       | 22.80        | 23.86                       | 49.83        | 63.14        | 3.83                      |
| IMTL        | 39.35                   | 65.60        | 0.54               | 0.23        | 26.02                       | 21.19        | 26.20                       | 53.13        | 66.24        | -0.59                     |
| MoCo        | 40.30                   | 66.07        | 0.56               | 0.22        | 26.67                       | 21.83        | 25.61                       | 51.78        | 64.85        | 0.16                      |
| CAGrad      | 39.79                   | 65.49        | 0.55               | 0.23        | 26.31                       | 21.58        | 25.61                       | 52.36        | 65.58        | 0.29                      |
| NashMTL     | 40.13                   | 65.93        | 0.53               | 0.22        | 25.26                       | 20.08        | 28.40                       | 55.47        | 68.15        | -4.04                     |
| FAMO        | 38.88                   | 64.90        | 0.55               | 0.22        | 25.06                       | 19.57        | 29.21                       | 56.61        | 68.98        | -4.10                     |
| FairGrad    | 39.74                   | 66.01        | 0.54               | 0.22        | 24.84                       | 19.60        | 29.26                       | 56.58        | 69.16        | -4.66                     |
| FairGrad-R  | 39.21                   | 65.14        | 0.56               | 0.23        | 24.56                       | 19.04        | 30.30                       | 57.78        | 70.04        | -4.77                     |
| F-CAGrad    | <b>40.93</b>            | <b>66.68</b> | <b>0.53</b>        | 0.22        | 25.43                       | 20.39        | 27.99                       | 54.82        | 67.56        | -3.78                     |
| COST        | 38.06                   | 64.71        | 0.54               | 0.23        | 24.47                       | 18.80        | 30.84                       | <b>58.25</b> | <b>70.30</b> | -5.39                     |
| CPI (Ours)  | 37.84                   | 64.79        | 0.54               | <b>0.22</b> | <b>24.43</b>                | <b>18.89</b> | <b>30.68</b>                | 58.08        | 70.26        | <b>-5.82</b>              |

These results demonstrate that CPI generalizes well to complex multi-task settings with diverse output structures and loss geometries.

### 5.3.2 Plug-and-Play Evaluation.

To evaluate the plug-and-play compatibility of CPI, we integrate it with several representative MTL methods, including CAGrad, FAMO, and FairGrad, without modifying their original objectives or update rules. Table 4 reports the results on the Cityscapes dataset, with results on additional datasets provided in the Appendix A.3.

On Cityscapes, CPI reduces  $\Delta m\%$  across different optimizers, indicating improved trade-offs between segmentation and depth estimation. CPI-CAGrad achieves a lower  $\Delta m\%$  than both CAGrad and its reproduced result, and CPI-FairGrad further improves upon FairGrad, yielding the lowest  $\Delta m\%$  among all methods. CPI-FAMO also remains competitive relative to its baseline. Together with similar trends observed on other datasets, these results demonstrate that CPI can be seamlessly combined with diverse MTL methods.

### 5.3.3 Paired Seed-wise Analysis.

Figure 4 presents a paired seed-wise comparison of  $\Delta m\%$  for FairGrad and CAGrad with and without CPI on Cityscapes and NYU-v2, using five random seeds. For each seed, the original method and its CPI-initialized counterpart are trained under otherwise identical settings. Across the five paired runs, CPI improves or maintains  $\Delta m\%$  in most cases, indicating that the gains are not driven by a few favorable seeds. The more concentrated seed-wise profiles further suggest that CPI helps mitigate initialization-induced instability.

Table 4: Plug-and-play results on *Cityscapes* (2-task) dataset.

| Method              | Seg. $\uparrow$ |              | Depth $\downarrow$ |              | $\Delta m\%$ $\downarrow$ |
|---------------------|-----------------|--------------|--------------------|--------------|---------------------------|
|                     | mIoU            | Pix. Acc.    | Abs.               | Rel.         |                           |
| CAGrad              | 75.16           | 93.48        | 0.0141             | 37.60        | 11.58                     |
| CAGrad-R            | 75.38           | 93.57        | 0.0142             | 38.38        | 12.30                     |
| <b>CPI-CAGrad</b>   | <b>75.41</b>    | <b>93.58</b> | <b>0.0143</b>      | <b>36.47</b> | <b>10.88</b>              |
| FAMO                | 74.54           | 93.29        | 0.0145             | 32.59        | <b>8.13</b>               |
| FAMO-R              | 73.50           | 93.06        | 0.0152             | 33.03        | 10.39                     |
| <b>CPI-FAMO</b>     | <b>74.88</b>    | <b>93.36</b> | <b>0.0137</b>      | <b>36.48</b> | <b>9.83</b>               |
| FairGrad            | 75.72           | 93.68        | 0.0134             | 32.25        | 5.18                      |
| FairGrad-R          | 74.61           | 93.36        | 0.0131             | 31.47        | 4.23                      |
| <b>CPI-FairGrad</b> | <b>73.76</b>    | <b>92.98</b> | <b>0.0133</b>      | <b>28.94</b> | <b>2.74</b>               |

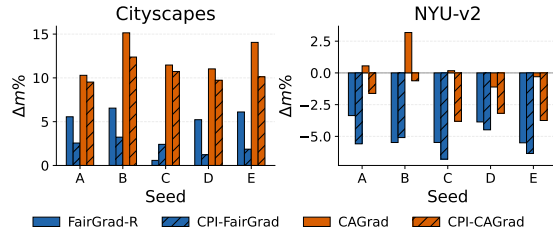


Figure 4: Seed-wise  $\Delta m\%$  of FairGrad-R and CAGrad with and without CPI on Cityscapes and NYU-v2. Solid bars denote the original methods, while hatched bars denote the corresponding methods initialized with CPI.

## 6 Conclusion

In this work, we first systematically revisit the role of parameter initialization in MTL and demonstrate that initialization constitutes a critical yet underexplored source of performance variability. By analyzing the local geometry of the multi-task loss landscape at initialization, we find that curvature is closely related to subsequent optimization behavior. Guided by this insight, we propose CPI, an optimizer-agnostic plug-in method that biases the initialization toward empirically favorable nearby regions while controlling first-order loss effects before standard training. Empirical results across multiple benchmarks demonstrate that CPI leads to consistent performance improvements, motivating further investigation into initialization-aware MTL design.

## References

- Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE TPAMI*, 39(12):2481–2495, 2017.
- Hao Ban and Kaiyi Ji. Fair resource allocation in multi-task learning. *arXiv:2402.15638*, 2024.
- Linxiao Cao, Mianzimei Yang, Zhipeng Zhou, Hong Xie, Defu Lian, and Menglin Yang. Towards calibrated gradient-based multi-task learning. In *Proceedings of CVPR*, pages 5127–5136, 2026.
- Pratik Chaudhari, Anna Choromanska, Stefano Soatto, Yann LeCun, Carlo Baldassi, Christian Borgs, Jennifer Chayes, Levent Sagun, and Riccardo Zecchina. Entropy-sgd: Biasing gradient descent into wide valleys. *Journal of Statistical Mechanics: Theory and Experiment*, 2019(12):124018, 2019.
- Shijie Chen, Yu Zhang, and Qiang Yang. Multi-task learning in natural language processing: An overview. *ACM Computing Surveys*, 56(12):1–32, 2024.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of ICML*, pages 794–803. PMLR, 2018.
- Zhao Chen, Jiquan Ngiam, Yanping Huang, Thang Luong, Henrik Kretschmar, Yuning Chai, and Dragomir Anguelov. Just pick a sign: Optimizing deep multitask models with gradient sign dropout. In *Proceedings of NeurIPS*, pages 2039–2050, 2020.
- Zitian Chen, Yikang Shen, Mingyu Ding, Zhenfang Chen, Hengshuang Zhao, Erik G Learned-Miller, and Chuang Gan. Mod-squad: Designing mixtures of experts as modular multi-task learners. In *Proceedings of CVPR*, pages 11828–11837, 2023.
- Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of CVPR*, pages 3213–3223, 2016.
- Michael Crawshaw. Multi-task learning with deep neural networks: A survey. *arXiv:2009.09796*, 2020.
- Zhiwen Fan, Rishov Sarkar, Ziyu Jiang, Tianlong Chen, Kai Zou, Yu Cheng, Cong Hao, Zhangyang Wang, et al. M<sup>3</sup>vit: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design. In *Proceedings of NeurIPS*, pages 28441–28457, 2022.
- Heshan Fernando, Han Shen, Miao Liu, Subhajt Chaudhury, Keerthiram Murugesan, and Tianyi Chen. Mitigating gradient bias in multi-objective learning: A provably convergent approach. In *Proceedings of ICLR*, 2023.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv:1903.02428*, 2019.
- Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *arXiv:2010.01412*, 2020.

- Stanislav Fort and Adam Scherlis. The goldilocks zone: Towards better understanding of neural network loss landscapes. *arXiv:1807.02581*, 2018.
- Yuan Gao, Jiayi Ma, Mingbo Zhao, Wei Liu, and Alan L Yuille. Nddr-cnn: Layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction. In *Proceedings of CVPR*, pages 3205–3214, 2019.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS*, pages 249–256, 2010.
- Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. *arXiv:2110.04366*, 2021.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of CVPR*, pages 1026–1034, 2015.
- Falk Heuer, Sven Mantowsky, Saqib Bukhari, and Georg Schneider. Multitask-centernet (mcn): Efficient and diverse multitask learning using an anchor free approach. In *Proceedings of ICCV*, pages 997–1005, 2021.
- Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of CVPR*, pages 7482–7491, 2018.
- Baijiong Lin, Feiyang Ye, Yu Zhang, and Ivor W Tsang. Reasonable effectiveness of random weighting: A litmus test for multi-task learning. *arXiv:2111.10603*, 2021.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Proceedings of NeurIPS*, pages 18878–18890, 2021a.
- Bo Liu, Yihao Feng, Peter Stone, and Qiang Liu. Famo: Fast adaptive multitask optimization. In *Proceedings of NeurIPS*, pages 57226–57243, 2023.
- Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Proceedings of NeurIPS*, pages 1950–1965, 2022.
- Liyang Liu, Yi Li, Zhanghui Kuang, J Xue, Yimin Chen, Wenming Yang, Qingmin Liao, and Wayne Zhang. Towards impartial multi-task learning. In *Proceedings of ICLR*, 2021b.
- Shikun Liu, Edward Johns, and Andrew J Davison. End-to-end multi-task learning with attention. In *Proceedings of CVPR*, pages 1871–1880, 2019.
- Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of ICCV*, pages 3730–3738, 2015.
- Dmytro Mishkin and Jiri Matas. All you need is a good init. *arXiv:1511.06422*, 2015.
- Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game. *arXiv:2202.01017*, 2022.
- Hoang Phan, Lam Tran, Quyen Tran, Ngoc Tran, Tuan Truong, Qi Lei, Nhat Ho, Dinh Phung, and Trung Le. Beyond losses reweighting: Empowering multi-task learning via the generalization perspective. In *Proceedings of ICCV*, pages 2440–2450, 2025.
- Xiaohan Qin, Xiaoxing Wang, Ning Liao, and Junchi Yan. Ntkmtl: Mitigating task imbalance in multi-task learning from neural tangent kernel perspective. *arXiv preprint arXiv:2510.18258*, 2025a.
- Xiaohan Qin, Xiaoxing Wang, and Junchi Yan. Towards consistent multi-task learning: Unlocking the potential of task-specific parameters. In *Proceedings of CVPR*, 2025b.
- Raghuathan Ramakrishnan, Pavlo O Dral, Matthias Rupp, and O Anatole Von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific data*, 1(1):1–7, 2014.

- Levent Sagun, Leon Bottou, and Yann LeCun. Eigenvalues of the hessian in deep learning: Singularity and beyond. *arXiv:1611.07476*, 2016.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. In *Proceedings of NeurIPS*, 2018.
- Jiayi Shen, Qi Wang, Zehao Xiao, Nanne Van Noord, and Marcel Worring. Go4align: Group optimization for multi-task alignment. In *Proceedings of NeurIPS*, 2024.
- Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor segmentation and support inference from rgb-d images. In *Proceedings of ECCV*, pages 746–760, 2012.
- Artem Vysogorets, Anna Dawid, and Julia Kempe. Deconstructing the goldilocks zone of neural network initialization. *arXiv:2402.03579*, 2024.
- Zedong Wang, Siyuan Li, and Dan Xu. Rep-mtl: Unleashing the power of representation-level task saliency for multi-task learning. In *Proceedings of ICCV*, 2025.
- Yongxin Yang and Timothy Hospedales. Deep multi-task representation learning: A tensor factorisation approach. *arXiv:1605.06391*, 2016.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Proceedings of NeurIPS*, pages 5824–5836, 2020.
- Yu Zhang and Qiang Yang. A survey on multi-task learning. *IEEE TKDE*, 34(12):5586–5609, 2021.
- Zipeng Zhou, Liu Liu, Peilin Zhao, and Wei Gong. Pareto deep long-tailed recognition: A conflict-averse solution. In *Proceedings of ICLR*, 2024.
- Zipeng Zhou, Liu Liu, Peilin Zhao, and Wei Gong. Injecting imbalance sensitivity for multi-task learning. *arXiv:2503.08006*, 2025a.
- Zipeng Zhou, Ziqiao Meng, Pengcheng Wu, Peilin Zhao, and Chunyan Miao. Continual optimization with symmetry teleportation for multi-task learning. *arXiv:2503.04046*, 2025b.
- Zipeng Zhou, Ziqiao Meng, Pengcheng Wu, Peilin Zhao, and Chunyan Miao. Exploring tradeoffs through mode connectivity for multi-task learning. In *Proceedings of NeurIPS*, 2025c.

# Appendix

## Contents

### Appendix A: Experimental Details

- Appendix A.1: Implementation Details
- Appendix A.2: Efficiency-Oriented Initialization Design
- Appendix A.3: Additional Plug-and-Play Verification
- Appendix A.4: Ablation Results
- Appendix A.5: Curvature Analysis at Initialization
- Appendix A.6: Efficiency Evaluation
- Appendix A.7: Hyperparameter Sensitivity
- Appendix A.8: Experimental Results with Error Bars
- Appendix A.9: Relation to Meta-Learned Initialization
- Appendix A.10: Comparison with Sharpness-Aware Initialization
- Appendix A.11: Additional Initialization Baselines

### Appendix B: Proofs

- Appendix B.1: Proof of Theorem 3.2
- Appendix B.2: Connecting Directional Curvature Probes to Curvature-Controlled Regions

### Appendix C: Limitations

## A Experimental Details

### A.1 Implementation Details

#### A.1.1 Dataset and Task Descriptions

We evaluate CPI across three supervised MTL scenarios: large-scale classification, regression, and dense prediction—each presenting unique optimization challenges.

**Image-Level Multi-Task Classification.** CelebA [Liu et al., 2015] is a large-scale face attribute dataset containing over 200K celebrity images, each annotated with 40 binary attributes (e.g., Smiling, Wavy Hair, Mustache). This benchmark is framed as a 40-task binary classification problem and is commonly used to evaluate the scalability of MTL methods.

**Multi-Task Regression.** QM9 [Ramakrishnan et al., 2014] consists of 134K organic molecules represented as graphs. Each molecule is annotated with 11 real-valued regression targets corresponding to quantum chemical properties with different numerical scales. This task is used to test the ability of MTL methods to handle regression with varying magnitudes and distributions.

**Dense Prediction.** NYU-v2 [Silberman et al., 2012] includes 1,449 indoor RGB-D images with three pixel-wise tasks: 13-class semantic segmentation, depth estimation, and surface normal prediction. Cityscapes [Cordts et al., 2016] contains 5,000 urban street scenes with two pixel-wise tasks: 7-class segmentation and depth estimation. These datasets evaluate the robustness of MTL methods on dense prediction with heterogeneous task types and output structures.

**Dataset and Asset Licenses.** We use only publicly available benchmark datasets and standard open-source implementations. CelebA is used for academic research following its official terms of use. QM9 is distributed as part of public molecular benchmark resources and is commonly accessed through PyTorch Geometric. NYU-v2 and Cityscapes are used under their respective research/non-commercial dataset terms. For baseline implementations, we rely on publicly available code released by the corresponding authors when available, and cite the original papers for all datasets, architectures, and methods used in our experiments. Our use of these assets is limited to research evaluation and follows their intended benchmark usage.

#### A.1.2 Training Details

All experiments are conducted using PyTorch 2.7.1 and CUDA 12.4 on NVIDIA A40 GPUs (48GB). We closely follow experimental settings from prior work [Liu et al., 2023, Navon et al., 2022], with minor adjustments for consistency.

- CelebA: A 9-layer CNN is used as the shared backbone, with a separate linear classification head for each attribute. Models are trained using Adam optimizer with an initial learning rate of  $3 \times 10^{-4}$ , batch size 256, and for 15 epochs.
- QM9: We adopt the official PyTorch Geometric implementation [Fey and Lenssen, 2019], using the standard 110K/10K/10K split for training, validation, and testing. Models are trained for 300 epochs with batch size 120. The initial learning rate is set to  $1 \times 10^{-3}$ , and a scheduler reduces the learning rate upon validation stagnation.
- NYU-v2: We use the MTAN architecture [Liu et al., 2019] based on SegNet [Badrinarayanan et al., 2017], with task-specific attention modules. Models are trained for 200 epochs with a batch size of 2. The learning rate is  $1 \times 10^{-4}$  for the first 100 epochs and halved afterward.
- Cityscapes: We use the same architecture and training schedule as for NYU-v2, except with a batch size of 8 due to higher image resolution. The total training duration is also 200 epochs.

**Curvature Estimation Details** During the curvature-guided initialization, we estimate task-wise curvature using a small number of stochastic probe directions (5 per task) to ensure computational efficiency. In addition, a small constant  $\delta = 10^{-4}$  is used in the update rule to ensure numerical stability.

Table 5: Plug and play results on *NYU-v2* (3-task) dataset. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method             | Segmentation $\uparrow$ |           | Depth $\downarrow$ |           | Surface Normal              |        |                             |       |       | $\Delta m\%$ $\downarrow$ |
|--------------------|-------------------------|-----------|--------------------|-----------|-----------------------------|--------|-----------------------------|-------|-------|---------------------------|
|                    | mIoU                    | Pix. Acc. | Abs. Err.          | Rel. Err. | Angle Distance $\downarrow$ |        | Within $t^\circ$ $\uparrow$ |       |       |                           |
|                    |                         |           |                    |           | Mean                        | Median | 11.25                       | 22.5  | 30    |                           |
| CAGrad             | 39.79                   | 65.49     | 0.55               | 0.23      | 26.31                       | 21.58  | 25.61                       | 52.36 | 65.58 | 0.29                      |
| CAGrad- <b>R</b>   | 41.68                   | 67.72     | 0.54               | 0.22      | 26.48                       | 22.20  | 24.90                       | 51.15 | 64.78 | 0.17                      |
| CPI-CAGrad         | 40.16                   | 66.36     | 0.53               | 0.22      | 25.87                       | 20.93  | 26.83                       | 53.63 | 66.63 | <b>-2.02</b>              |
| FAMO               | 38.88                   | 64.90     | 0.55               | 0.22      | 25.06                       | 19.57  | 29.21                       | 56.61 | 68.98 | -4.10                     |
| FAMO- <b>R</b>     | 36.43                   | 63.37     | 0.56               | 0.23      | 24.97                       | 19.20  | 30.06                       | 57.34 | 69.37 | -3.49                     |
| CPI-FAMO           | 36.58                   | 63.72     | 0.56               | 0.22      | 24.88                       | 18.99  | 30.29                       | 57.80 | 69.74 | <b>-4.11</b>              |
| FairGrad           | 39.74                   | 66.01     | 0.54               | 0.22      | 24.84                       | 19.60  | 29.26                       | 56.58 | 69.16 | -4.66                     |
| FairGrad- <b>R</b> | 39.21                   | 65.14     | 0.56               | 0.23      | 24.56                       | 19.04  | 30.30                       | 57.78 | 70.04 | -4.77                     |
| CPI-FairGrad       | 37.84                   | 64.79     | 0.54               | 0.22      | 24.43                       | 18.89  | 30.68                       | 58.08 | 70.26 | <b>-5.82</b>              |

**Experimental Protocol** Following common practice in prior MTL studies [Liu et al., 2023, Navon et al., 2022], we report results averaged over three random seeds. This protocol enables a fair comparison with existing MTL baselines under the same evaluation budget. We apply the same random seeds across all methods—including our method and all baseline approaches such as FAMO-R, FairGrad-R, to ensure a fair comparison under identical training conditions.

**Parameter Perturbation Protocol.** Given a fixed random seed, we first obtain a standard initialization of the model parameters. We then generate multiple perturbed initializations by adding random noise vectors to the initial parameters. Specifically, we sample 10 random directions and normalize each direction to have an  $\ell_2$  norm of  $10^{-4}$ , and add them to the original initialization. Each perturbed initialization is used to train the model independently under identical training settings, including data order, optimization hyperparameters, and model architecture.

## A.2 Efficiency-Oriented Initialization Design

To improve the efficiency of the initialization stage, we incorporate two complementary mechanisms: early stopping to avoid unnecessary updates, and gradient-similarity-based task grouping to reduce redundant computation for datasets with many tasks.

**Early stopping.** Let  $\Delta\theta_t$  denote the initialization update at step  $t$ . The initialization procedure is run for at most  $T_{\max}$  (one epoch) steps and terminated early if the update magnitude  $\|\Delta\theta_t\|_2$  remains below a threshold  $\varepsilon$  for several consecutive steps. This criterion reflects diminishing returns once the curvature-guided updates have effectively stabilized, and is applied consistently across all datasets and random seeds.

**Gradient-based task grouping.** For CelebA and QM9, we further reduce initialization overhead by grouping tasks based on gradient similarity at initialization. Specifically, we compute task-wise gradients  $g_i = \nabla_{\theta} \mathcal{L}_i(\theta)$  and construct a similarity matrix using cosine similarity  $S_{ij} = \langle g_i, g_j \rangle / (\|g_i\|_2 \|g_j\|_2)$ . Spectral clustering is then applied to  $S$  to obtain task groups, and tasks within the same group share computations during initialization. This grouping is performed once per random seed and only affects the initialization stage.

Both mechanisms operate exclusively during initialization and introduce no additional cost during training. Their overhead is included in the reported initialization time.

Table 6: Plug-and-play results on *CelebA* (40-task) and *QM9* (11-task) datasets. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method                                | CelebA                    | QM9                       |
|---------------------------------------|---------------------------|---------------------------|
|                                       | $\Delta m\%$ $\downarrow$ | $\Delta m\%$ $\downarrow$ |
| CAGrad [Liu et al., 2021a]            | 2.48                      | 112.8                     |
| CPI-CAGrad                            | <b>1.99</b>               | <b>109.4</b>              |
| FAMO [Liu et al., 2023]               | 1.21                      | <b>58.5</b>               |
| FAMO- <b>R</b> [Liu et al., 2023]     | 0.69                      | 68.6                      |
| CPI-FAMO                              | <b>0.56</b>               | 63.2                      |
| FairGrad [Ban and Ji, 2024]           | <b>0.37</b>               | 57.9                      |
| FairGrad- <b>R</b> [Ban and Ji, 2024] | 1.09                      | 59.9                      |
| CPI-FairGrad                          | 0.52                      | <b>55.2</b>               |

### A.3 Additional Plug and Play Verification

Tables 5 and 6 report plug-and-play results on three additional datasets: NYU-v2, CelebA, and QM9. Consistent with the observations on Cityscapes, incorporating CPI generally leads to lower  $\Delta m\%$  across different MTL optimizers and task configurations. On NYU-v2, CPI improves the overall multi-task trade-off for CAGrad, FAMO, and FairGrad, achieving the lowest  $\Delta m\%$  within each optimizer family. Similar trends are observed on CelebA and QM9, where CPI consistently matches or improves upon the corresponding baselines. These results further support that CPI serves as a robust and broadly applicable plug-and-play initialization strategy across datasets with varying task numbers and characteristics.

### A.4 Ablation Results

We conduct ablation studies to isolate the role of first-order loss control in the proposed curvature-guided initialization. Specifically, we evaluate a variant of CPI that removes the first-order projection step, thereby allowing curvature-guided updates to induce uncontrolled first-order loss variations. All variants share the same update framework and are evaluated under identical experimental settings.

As shown in Table 7, removing first-order projection leads to degraded stability and inferior overall performance, highlighting the necessity of controlling first-order loss effects when applying curvature-guided updates.

### A.5 Curvature Analysis at Initialization

We analyze the curvature structure at initialization by probing the Hessian along multiple random directions. Specifically, for each task, we sample 500 probe directions and estimate the corresponding directional curvature values, which characterize the local second-order geometry around the current parameters. These estimates provide stochastic directional measurements of the local second-order geometry.

Figure 5 visualizes the resulting curvature distributions using violin plots. Before initialization, the two tasks exhibit markedly different curvature profiles, with their mean curvature even having opposite signs in some cases. After applying the proposed curvature-guided initialization, the task-wise curvature distributions become more aligned, with their means converging toward a consistent and moderately positive regime. This indicates that the initialization effectively reshapes the local geometry to reduce task-specific curvature discrepancies.

### A.6 Efficiency Evaluation

Table 7: Ablation results on *Cityscapes* (2-task) dataset. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method                         | Segmentation $\uparrow$ |              | Depth $\downarrow$ |              | $\Delta m\%$ $\downarrow$ |
|--------------------------------|-------------------------|--------------|--------------------|--------------|---------------------------|
|                                | mIoU                    | Pix. Acc.    | Abs. Err.          | Rel. Err.    |                           |
| CPI w/o First-Order Projection | 75.28                   | 93.53        | 0.0134             | 31.10        | 4.17                      |
| <b>CPI (Ours)</b>              | <b>73.76</b>            | <b>92.98</b> | <b>0.0133</b>      | <b>28.94</b> | <b>2.74</b>               |

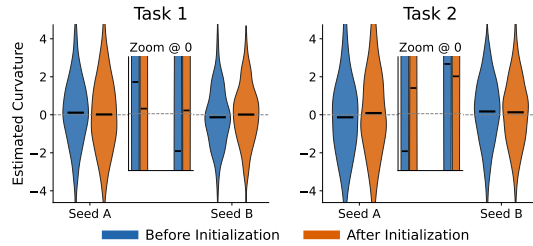


Figure 5: Violin plots of task-wise curvature distributions before and after initialization. For each task, the full distributions are shown for Seed A and Seed B, with a zoomed view highlighting the region around zero curvature. Horizontal bars indicate the mean curvature, and the dashed line marks zero curvature.

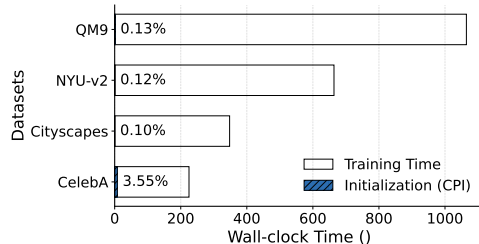


Figure 6: Initialization overhead of CPI relative to full training time across different datasets. Hollow bars denote the total wall-clock training time, while solid bars indicate the additional initialization time introduced by CPI. The percentage labels report the ratio between initialization and training time.

Table 8: Hyperparameter  $\rho$  analysis results on *Cityscapes* (2-task) dataset. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method                      | Segmentation $\uparrow$ |           | Depth $\downarrow$ |           | $\Delta m\%$ $\downarrow$ |
|-----------------------------|-------------------------|-----------|--------------------|-----------|---------------------------|
|                             | mIoU                    | Pix. Acc. | Abs. Err.          | Rel. Err. |                           |
| FairGrad (Baseline)         | 75.72                   | 93.68     | 0.0134             | 32.25     | 5.18                      |
| CPI-FairGrad ( $\rho=0.3$ ) | 74.88                   | 93.45     | 0.0132             | 31.32     | 4.14                      |
| CPI-FairGrad ( $\rho=0.5$ ) | 74.15                   | 93.30     | 0.0130             | 29.77     | 2.79                      |
| CPI-FairGrad ( $\rho=0.7$ ) | 73.76                   | 92.98     | 0.0133             | 28.94     | <b>2.74</b>               |
| CPI-FairGrad ( $\rho=0.9$ ) | 74.64                   | 93.33     | 0.0135             | 30.46     | 4.24                      |

We evaluate the efficiency of the proposed CPI method by measuring the wall-clock time of its initialization stage across multiple datasets. All results are averaged over three random seeds under identical hardware and hyperparameter settings, and we focus exclusively on the additional cost introduced by initialization. As illustrated in the Figure 6, CPI incurs the lowest initialization overhead on Cityscapes and the highest on CelebA, primarily due to the larger number of tasks and the associated cost of task-wise curvature estimation. Importantly, the initialization overhead constitutes only a small fraction of the total training time across all datasets, indicating that CPI achieves consistent performance improvements with minimal additional computational cost.

### A.7 Hyperparameter Sensitivity

We analyze the sensitivity of CPI to the hyperparameter  $\rho$  on the Cityscapes dataset, which controls the relative weighting between correcting negative curvature and suppressing excessively large positive curvature. Table 8 reports the results under different  $\rho$  values.

Overall, CPI is moderately robust to the choice of  $\rho$ , achieving its best performance when  $\rho$  lies in the range of 0.5–0.7. In particular, intermediate values ( $\rho \in [0.5, 0.7]$ ) achieve a favorable balance between segmentation and depth estimation, resulting in consistently lower  $\Delta m$ . When  $\rho$  is set to smaller or larger values, performance degrades slightly, yet remains competitive compared to the baseline, indicating that over-emphasizing either curvature correction or positive curvature suppression alone is suboptimal. Based on these observations, we set  $\rho = 0.7$  as the default value.

### A.8 Experimental Results with Error Bars

We report detailed error bars for CPI on CelebA/QM9, Cityscapes, and NYU-v2 in Tables 9, 10, and 12, respectively. To provide a more complete comparison, we further reproduce several representative baseline methods under the same experimental protocol and summarize their  $\Delta m\%$  error bars in Table 11. Across the reproduced comparisons, CPI achieves the best mean  $\Delta m\%$  and shows competitive seed-wise variability.

Table 9: Results with error bars on *CelebA* (40-task) and *QM9* (11-task). Each experiment is repeated 3 times with different random seeds.

| Method     | CelebA                    | QM9                       |
|------------|---------------------------|---------------------------|
|            | $\Delta m\%$ $\downarrow$ | $\Delta m\%$ $\downarrow$ |
| CPI (mean) | 0.52                      | 55.2                      |
| CPI (std)  | $\pm 0.2908$              | $\pm 12.9274$             |

Table 10: Results with error bars on *Cityscapes* (2-task). Each experiment is repeated 3 times with different random seeds.

| Method     | Seg. $\uparrow$ |              | Depth $\downarrow$ |              | $\Delta m\%$ $\downarrow$ |
|------------|-----------------|--------------|--------------------|--------------|---------------------------|
|            | mIoU            | Pix. Acc.    | Abs.               | Rel.         |                           |
| CPI (mean) | 73.76           | 92.98        | 0.0133             | 28.94        | 2.74                      |
| CPI (std)  | $\pm 0.0180$    | $\pm 0.0067$ | $\pm 0.0009$       | $\pm 2.5091$ | $\pm 0.4378$              |

Table 11: Results with error bars on Cityscapes, QM9, and NYU-v2. We report mean and standard deviation over multiple random seeds. Methods marked with “-R” denote reproduced baselines under the same experimental protocol. Lower  $\Delta m\%$  is better.

| Method     | Cityscapes                        | QM9                                | NYU-v2                             |
|------------|-----------------------------------|------------------------------------|------------------------------------|
| CAGrad-R   | 12.30 $\pm$ 2.5305                | 112.8 $\pm$ 8.7772                 | 0.17 $\pm$ 1.6325                  |
| FAMO-R     | 10.39 $\pm$ 3.7770                | 68.6 $\pm$ 22.5750                 | -3.49 $\pm$ 0.5914                 |
| FairGrad-R | 4.23 $\pm$ 3.1988                 | 59.8 $\pm$ 17.4936                 | -4.77 $\pm$ 1.2235                 |
| CPI        | <b>2.74<math>\pm</math>0.4378</b> | <b>55.2<math>\pm</math>12.9274</b> | <b>-5.82<math>\pm</math>0.8734</b> |

Table 12: Results on *NYU-v2* (3-task) dataset with standard deviation. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method     | Segmentation $\uparrow$ |              | Depth $\downarrow$ |              | Surface Normal              |             |                             |              |              | $\Delta m\%$ $\downarrow$ |
|------------|-------------------------|--------------|--------------------|--------------|-----------------------------|-------------|-----------------------------|--------------|--------------|---------------------------|
|            | mIoU                    | Pix. Acc.    | Abs. Err.          | Rel. Err.    | Angle Distance $\downarrow$ |             | Within $t^\circ$ $\uparrow$ |              |              |                           |
|            |                         |              |                    |              | Mean                        | Median      | 11.25                       | 22.5         | 30           |                           |
| CPI (mean) | 37.84                   | 64.79        | 0.54               | 0.22         | 24.43                       | 18.89       | 30.68                       | 58.08        | 70.26        | -5.82                     |
| CPI (std)  | $\pm 0.0187$            | $\pm 0.0136$ | $\pm 0.0097$       | $\pm 0.0067$ | $\pm 0.2799$                | $\pm 0.321$ | $\pm 0.0052$                | $\pm 0.0071$ | $\pm 0.0058$ | $\pm 0.8734$              |

## A.9 Relation to Meta-Learned Initialization

CPI may appear superficially related to meta-learned initialization methods, since both seek favorable starting parameters for subsequent training. However, their objectives and optimization procedures are fundamentally different. Meta-learned initialization typically assumes a task distribution and optimizes an initialization through an inner-outer loop, where the inner loop adapts the parameters to sampled tasks and the outer-loop meta-objective evaluates post-adaptation performance. CPI contains none of these components: it does not rely on a task distribution, perform inner-loop adaptation, or optimize post-adaptation performance.

Instead, CPI is a single-stage pre-training/warm-up procedure that directly shapes the local geometry of the multi-task loss landscape at initialization. It steers parameters toward regions with positive and moderate task-wise curvature, thereby improving early-stage optimization stability across tasks, consistent with the theoretical analysis in Sec. 3.3. CPI is therefore better understood as a geometry-aware initialization strategy for MTL rather than as a meta-learning method.

Table 13: Comparison with a MAML-style meta-learned initialization baseline. Results are reported as mean  $\pm$  standard deviation over multiple seeds. Lower  $\Delta m\%$  is better.

| Method        | CelebA              | NYU-v2               |
|---------------|---------------------|----------------------|
| FairGrad-R    | $1.0884 \pm 1.1513$ | $-4.7749 \pm 1.2235$ |
| MAML+FairGrad | $2.0433 \pm 1.0629$ | $-1.3267 \pm 1.2783$ |
| CPI-FairGrad  | $0.5200 \pm 0.2908$ | $-5.8200 \pm 0.8734$ |

To further distinguish CPI from meta-learned initialization, we compare CPI with a MAML-style initialization baseline combined with FairGrad. As shown in Table 13, MAML+FairGrad does not reproduce the improvements achieved by CPI. On CelebA, MAML+FairGrad performs worse than the FairGrad-R baseline, whereas CPI achieves the best result. On NYU-v2, MAML+FairGrad improves over FairGrad-R but remains substantially worse than CPI. These results indicate that CPI’s gains do not simply come from learning a better initialization through a generic meta-learning procedure; rather, its task-wise curvature-guided objective is crucial.

## A.10 Comparison with Sharpness-Aware Initialization

We further compare CPI with a sharpness-based initialization baseline combined with FairGrad. While sharpness-aware methods [Foret et al., 2020] are also motivated by loss-landscape geometry, their objective differs from CPI: they typically penalize locally sharp regions or encourage flat minima, whereas CPI explicitly steers the initialization toward regions with positive and moderate task-wise curvature. This distinction is particularly important in MTL, where different tasks can exhibit heterogeneous curvature and gradient interactions.

Table 14: Comparison with a SAM-style initialization baseline. Results are reported as mean  $\pm$  standard deviation over multiple seeds. Lower  $\Delta m\%$  is better.

| Method       | CelebA              | NYU-v2               |
|--------------|---------------------|----------------------|
| FairGrad-R   | $1.0884 \pm 1.1513$ | $-4.7749 \pm 1.2235$ |
| SAM+FairGrad | $3.39 \pm 0.5794$   | $-4.72 \pm 0.8955$   |
| CPI-FairGrad | $0.5200 \pm 0.2908$ | $-5.8200 \pm 0.8734$ |

As shown in Table 14, SAM+FairGrad does not reproduce the gains achieved by CPI. On CelebA, SAM+FairGrad performs substantially worse than FairGrad-R and CPI. On NYU-v2, it is close to FairGrad-R but still worse than CPI. These results suggest that CPI’s improvement cannot be explained by generic sharpness reduction alone; instead, its advantage comes from the task-wise curvature-guided initialization objective.

### A.11 Additional Initialization Baselines

We further evaluate two initialization baselines on *Cityscapes* to better understand the source of the improvement brought by CPI.

*Gradient-based Initialization* constructs the initialization by simply performing standard gradient-based warm-up for the same number of steps used by our initialization procedure. This baseline controls for the possibility that the improvement comes from additional optimization steps rather than from the proposed curvature-aware initialization direction.

*Random-perturbed Initialization* replaces the proposed curvature-aware direction with a random perturbation of the same norm, while keeping the same projection and update rule. This baseline controls for whether the improvement can be attributed to a generic perturbation of the initialization, rather than to the specific curvature-aware direction estimated by CPI.

Table 15: Additional initialization baseline results on *Cityscapes* (2-task) dataset. Each experiment is repeated 3 times with different random seeds and the average is reported.

| Method                          | Segmentation $\uparrow$ |           | Depth $\downarrow$ |           | $\Delta m\%$ $\downarrow$ |
|---------------------------------|-------------------------|-----------|--------------------|-----------|---------------------------|
|                                 | mIoU                    | Pix. Acc. | Abs. Err.          | Rel. Err. |                           |
| Random-perturbed Initialization | 73.74                   | 93.15     | 0.0131             | 32.99     | 5.94                      |
| Gradient-based Initialization   | 74.49                   | 93.29     | 0.0134             | 30.27     | 3.89                      |
| CPI                             | 73.76                   | 92.98     | 0.0133             | 28.94     | <b>2.74</b>               |

As shown in Table 15, neither baseline matches the overall performance of CPI. Although the gradient-based initialization achieves comparable segmentation and absolute depth error, it substantially underperforms CPI in relative depth error and overall  $\Delta m\%$ . Similarly, the random-perturbed initialization does not recover the performance of CPI, suggesting that the benefit is not simply due to injecting a perturbation with a similar magnitude. These results indicate that the curvature-aware initialization direction, together with the projection step, is important for obtaining a better initialization.

## B Proofs

### B.1 Proof of Theorem 3.2

We prove Theorem 3.2 using local curvature control, an invariance argument, and standard smoothness and strong convexity properties.

#### B.1.1 Local Curvature Control

We first show that the curvature bounds at initialization extend to a neighborhood of  $\theta_0$ .

**Lemma B.1** (Local curvature bounds). *Assume that for all tasks  $t$ ,*

$$m_0 I \preceq H_t(\theta_0) \preceq L_0 I, \quad (16)$$

*and that each  $H_t$  is  $\rho$ -Lipschitz in operator norm. Define*

$$r := \frac{m_0}{2\rho}, \quad m_r := m_0 - \rho r = \frac{m_0}{2}, \quad L_r := L_0 + \rho r = L_0 + \frac{m_0}{2}. \quad (17)$$

*Then for all  $\theta$  satisfying  $\|\theta - \theta_0\| \leq r$  and all  $t$ ,*

$$m_r I \preceq H_t(\theta) \preceq L_r I. \quad (18)$$

*Proof.* By the Hessian Lipschitz assumption,

$$\|H_t(\theta) - H_t(\theta_0)\|_{op} \leq \rho \|\theta - \theta_0\|. \quad (19)$$

Hence,

$$\lambda_{\min}(H_t(\theta)) \geq \lambda_{\min}(H_t(\theta_0)) - \|H_t(\theta) - H_t(\theta_0)\|_{op} \geq m_0 - \rho \|\theta - \theta_0\|, \quad (20)$$

and

$$\lambda_{\max}(H_t(\theta)) \leq \lambda_{\max}(H_t(\theta_0)) + \|H_t(\theta) - H_t(\theta_0)\|_{op} \leq L_0 + \rho \|\theta - \theta_0\|. \quad (21)$$

If  $\|\theta - \theta_0\| \leq r = m_0/(2\rho)$ , then

$$\lambda_{\min}(H_t(\theta)) \geq m_0 - \rho r = m_r, \quad (22)$$

and

$$\lambda_{\max}(H_t(\theta)) \leq L_0 + \rho r = L_r. \quad (23)$$

Since  $F_\lambda(\theta) = \sum_{t=1}^T \lambda_t f_t(\theta)$  with  $\lambda \in \Delta_T$ , its Hessian satisfies

$$\nabla^2 F_\lambda(\theta) = \sum_{t=1}^T \lambda_t H_t(\theta). \quad (24)$$

Therefore, for all  $\theta \in \mathcal{B}(\theta_0, r)$ ,

$$m_r I \preceq \nabla^2 F_\lambda(\theta) \preceq L_r I. \quad (25)$$

□

### B.1.2 Invariance of the Curvature-Controlled Ball

We now show that gradient descent iterates remain in  $\mathcal{B}(\theta_0, r)$ .

Assume  $\theta_i \in \mathcal{B}(\theta_0, r)$ . By the fundamental theorem of calculus applied to  $g_\lambda = \nabla F_\lambda$ ,

$$g_\lambda(\theta_i) = g_\lambda(\theta_0) + \left( \int_0^1 \nabla^2 F_\lambda(\theta_0 + s(\theta_i - \theta_0)) ds \right) (\theta_i - \theta_0). \quad (26)$$

Define

$$A_i := \int_0^1 \nabla^2 F_\lambda(\theta_0 + s(\theta_i - \theta_0)) ds. \quad (27)$$

Since the entire segment lies in  $\mathcal{B}(\theta_0, r)$ , the local curvature bounds imply

$$m_r I \preceq A_i \preceq L_r I. \quad (28)$$

Using the gradient descent update  $\theta_{i+1} = \theta_i - \eta g_\lambda(\theta_i)$ , we obtain

$$\theta_{i+1} - \theta_0 = (I - \eta A_i)(\theta_i - \theta_0) - \eta g_\lambda(\theta_0). \quad (29)$$

For  $\eta \leq 1/L_r$ ,  $\|I - \eta A_i\|_{op} \leq 1 - \eta m_r$ , hence

$$\|\theta_{i+1} - \theta_0\| \leq (1 - \eta m_r) \|\theta_i - \theta_0\| + \eta \|g_\lambda(\theta_0)\|. \quad (30)$$

Using  $\|\theta_i - \theta_0\| \leq r$  and the initialization condition

$$\|g_\lambda(\theta_0)\| \leq \frac{m_r r}{2}, \quad (31)$$

we conclude

$$\|\theta_{i+1} - \theta_0\| \leq (1 - \eta m_r) r + \eta \frac{m_r r}{2} \leq r. \quad (32)$$

Since  $\theta_0 \in \mathcal{B}(\theta_0, r)$  holds trivially and

$$\theta_i \in \mathcal{B}(\theta_0, r) \Rightarrow \theta_{i+1} \in \mathcal{B}(\theta_0, r), \quad (33)$$

by induction we have

$$\theta_i \in \mathcal{B}(\theta_0, r), \quad \forall i \geq 1. \quad (34)$$

### B.1.3 Descent and Linear Convergence

Since all iterates remain in  $\mathcal{B}(\theta_0, r)$  where  $F_\lambda$  is  $L_r$ -smooth, the descent lemma gives

$$F_\lambda(\theta_{i+1}) \leq F_\lambda(\theta_i) - \eta \|g_\lambda(\theta_i)\|^2 + \frac{L_r \eta^2}{2} \|g_\lambda(\theta_i)\|^2. \quad (35)$$

For  $\eta \leq 1/L_r$ , this implies

$$F_\lambda(\theta_{i+1}) \leq F_\lambda(\theta_i) - \frac{\eta}{2} \|g_\lambda(\theta_i)\|^2. \quad (36)$$

Let  $\theta_\lambda^*$  denote the unique minimizer of  $F_\lambda$  in  $\mathcal{B}(\theta_0, r)$ . Since  $\nabla^2 F_\lambda(\theta) \succeq m_r I$  on this ball,  $F_\lambda$  is  $m_r$ -strongly convex there. Applying the fundamental theorem of calculus along the segment from  $\theta_\lambda^*$  to  $\theta_i$  yields

$$g_\lambda(\theta_i) = \left( \int_0^1 \nabla^2 F_\lambda(\theta_\lambda^* + s(\theta_i - \theta_\lambda^*)) ds \right) (\theta_i - \theta_\lambda^*). \quad (37)$$

Define

$$B_i := \int_0^1 \nabla^2 F_\lambda(\theta_\lambda^* + s(\theta_i - \theta_\lambda^*)) ds. \quad (38)$$

Since both  $\theta_i$  and  $\theta_\lambda^*$  lie in  $\mathcal{B}(\theta_0, r)$ ,

$$m_r I \preceq B_i \preceq L_r I. \quad (39)$$

Using  $\theta_{i+1} = \theta_i - \eta g_\lambda(\theta_i)$ , we obtain

$$\theta_{i+1} - \theta_\lambda^* = (I - \eta B_i)(\theta_i - \theta_\lambda^*). \quad (40)$$

For  $\eta \leq 1/L_r$ ,

$$\|\theta_{i+1} - \theta_\lambda^*\| \leq (1 - \eta m_r) \|\theta_i - \theta_\lambda^*\|. \quad (41)$$

Iterating yields

$$\|\theta_i - \theta_\lambda^*\| \leq (1 - \eta m_r)^i \|\theta_0 - \theta_\lambda^*\|. \quad (42)$$

This completes the proof.

## B.2 Connecting Directional Curvature Probes to Curvature-Controlled Regions

Theorem 3.2 shows that initialization points with positive and bounded task-wise curvature lead to stable early-stage MTL optimization. Since CPI relies on finitely many directional probes rather than exact Hessian eigendecomposition, this section clarifies the surrogate objective optimized by CPI and states an explicit transfer condition under which directional curvature progress can be related to a relaxed curvature-controlled region. This transfer condition is necessary because finite directional probes provide scalable approximations but cannot unconditionally characterize the full Hessian spectrum.

For constants  $0 < m < L$ , define the curvature-controlled region

$$\mathcal{R}_{m,L} = \{\theta : mI \preceq H_t(\theta) \preceq LI, \forall t \in \{1, \dots, T\}\}. \quad (43)$$

Equivalently,  $\theta \in \mathcal{R}_{m,L}$  if all eigenvalues of every task Hessian lie in  $[m, L]$ . We measure violation of this region by

$$V(\theta) = \sum_{t=1}^T \left[ (m - \lambda_{\min}(H_t(\theta)))_+ + (\lambda_{\max}(H_t(\theta)) - L)_+ \right], \quad (44)$$

where  $(a)_+ = \max(a, 0)$ . By construction,

$$V(\theta) = 0 \iff \theta \in \mathcal{R}_{m,L}. \quad (45)$$

For task  $t$  and a sampled unit direction  $h$ , CPI computes

$$K_t(\theta; h) = \frac{1}{2} (\kappa_t(\theta + \sigma h) + \kappa_t(\theta - \sigma h)), \quad (46)$$

and

$$s_t(\theta; h) = \frac{\kappa_t(\theta + \sigma h) - \kappa_t(\theta - \sigma h)}{2\sigma}. \quad (47)$$

The unprojected curvature-guided update is

$$u_t^{\text{curv}} = -\text{sign}(K_t) \text{sign}(s_t) \alpha_t(K_t) h, \quad (48)$$

where

$$\alpha_t(K_t) = \begin{cases} \rho |K_t|, & K_t < 0, \\ (1 - \rho) |K_t|, & K_t \geq 0. \end{cases} \quad (49)$$

We use the convention  $\text{sign}(0) = 0$ .

Define

$$\beta(K) = \begin{cases} \rho, & K < 0, \\ 1 - \rho, & K \geq 0, \end{cases} \quad \phi_\rho(K) = \frac{1}{2} \beta(K) K^2. \quad (50)$$

Then  $\alpha_t(K) = \beta(K) |K|$  and  $\phi'_\rho(K) = \beta(K) K$ , with  $\phi'_\rho(0) = 0$ .

**Lemma B.2 (alignment with the directional curvature surrogate).** Assume that  $\kappa_t$  is three-times continuously differentiable in a neighborhood of  $\theta$ , with uniformly bounded third directional derivatives. Then the unprojected CPI update satisfies

$$\langle \nabla_{\theta} \phi_{\rho}(K_t(\theta; h)), u_t^{\text{curv}} \rangle \leq -\beta(K_t)^2 K_t^2 |s_t| + C_{\kappa} \sigma^2 \beta(K_t)^2 K_t^2, \quad (51)$$

for some constant  $C_{\kappa} > 0$ .

**Proof.** For brevity, write  $K_t = K_t(\theta; h)$  and  $s_t = s_t(\theta; h)$ . Let  $q(a) = \kappa_t(\theta + ah)$ . Then

$$K_t(\theta; h) = \frac{q(\sigma) + q(-\sigma)}{2}, \quad s_t(\theta; h) = \frac{q(\sigma) - q(-\sigma)}{2\sigma}. \quad (52)$$

Moreover,

$$\langle \nabla_{\theta} K_t(\theta; h), h \rangle = \frac{q'(\sigma) + q'(-\sigma)}{2}. \quad (53)$$

By Taylor expansion around  $a = 0$ , and using the boundedness of the third directional derivative,

$$s_t(\theta; h) = q'(0) + O(\sigma^2), \quad (54)$$

and

$$\frac{q'(\sigma) + q'(-\sigma)}{2} = q'(0) + O(\sigma^2). \quad (55)$$

Therefore,

$$\langle \nabla_{\theta} K_t(\theta; h), h \rangle = s_t + O(\sigma^2). \quad (56)$$

Since  $\phi'_{\rho}(K_t) = \beta(K_t)K_t$ , we have

$$\nabla_{\theta} \phi_{\rho}(K_t) = \beta(K_t)K_t \nabla_{\theta} K_t. \quad (57)$$

Thus,

$$\begin{aligned} \langle \nabla_{\theta} \phi_{\rho}(K_t), u_t^{\text{curv}} \rangle &= \beta(K_t)K_t \langle \nabla_{\theta} K_t, -\text{sign}(K_t) \text{sign}(s_t) \alpha_t(K_t) h \rangle \\ &= -\beta(K_t)K_t \text{sign}(K_t) \text{sign}(s_t) \alpha_t(K_t) (s_t + O(\sigma^2)). \end{aligned} \quad (58)$$

Since  $K_t \text{sign}(K_t) = |K_t|$  and  $\alpha_t(K_t) = \beta(K_t)|K_t|$ , it follows that

$$\langle \nabla_{\theta} \phi_{\rho}(K_t), u_t^{\text{curv}} \rangle \leq -\beta(K_t)^2 K_t^2 |s_t| + C_{\kappa} \sigma^2 \beta(K_t)^2 K_t^2. \quad (59)$$

This proves the claim.

Lemma B.2 shows that the sign-based CPI update is a descent direction for the directional curvature surrogate explicitly estimated by CPI, up to finite-difference error. In the practical CPI update, we further control first-order loss effects by applying a task-wise projection before aggregation.

For each task  $t$ , let

$$g_t(\theta) = \nabla_{\theta} L_t(\theta). \quad (60)$$

Given the unprojected curvature-guided update  $u_t^{\text{curv}}$ , CPI constructs

$$\tilde{u}_t^{\text{curv}} = u_t^{\text{curv}} - \frac{\langle u_t^{\text{curv}}, g_t(\theta) \rangle}{\|g_t(\theta)\|_2^2 + \epsilon} g_t(\theta), \quad (61)$$

where  $\epsilon > 0$  is a small numerical constant. Therefore,

$$\langle g_t(\theta), \tilde{u}_t^{\text{curv}} \rangle = \frac{\epsilon}{\|g_t(\theta)\|_2^2 + \epsilon} \langle g_t(\theta), u_t^{\text{curv}} \rangle. \quad (62)$$

In particular, when  $\epsilon = 0$  and  $g_t(\theta) \neq 0$ , the task-wise projected update is exactly orthogonal to the corresponding task gradient. For  $\epsilon > 0$ , the self-induced first-order effect is attenuated by the factor  $\epsilon/(\|g_t(\theta)\|_2^2 + \epsilon)$ .

The final CPI update is obtained by aggregating the projected task-wise updates:

$$\Delta\theta = \sum_{t=1}^T \tilde{u}_t^{\text{curv}}. \quad (63)$$

For task  $t$ , the first-order variation induced by the aggregated update is

$$\langle g_t(\theta), \Delta\theta \rangle = \langle g_t(\theta), \tilde{u}_t^{\text{curv}} \rangle + \sum_{j \neq t} \langle g_t(\theta), \tilde{u}_j^{\text{curv}} \rangle. \quad (64)$$

Thus, the task-wise projection removes, up to the numerical residual caused by  $\epsilon$ , the direct first-order effect of each task-specific curvature update on its own task loss. The remaining terms correspond to cross-task first-order interactions introduced by aggregation.

The task-wise projection may reduce the curvature-surrogate descent established in Lemma B.2, since it removes from each  $u_t^{\text{curv}}$  the component aligned with the corresponding task gradient. In addition, translating directional curvature descent into progress on the spectral violation  $V$  requires the sampled probes to cover the curvature-violating eigenspaces. We make these requirements explicit below.

**Assumption B.3 (probe-to-spectrum transfer).** There exist constants  $c > 0$ ,  $C_{\text{fd}} \geq 0$ , and  $\epsilon_{\text{probe}} \geq 0$  such that, for the task-wise projected CPI update  $\Delta\theta = \sum_{t=1}^T \tilde{u}_t^{\text{curv}}$  and for some Clarke subgradient  $g_V(\theta) \in \partial V(\theta)$ ,

$$\mathbb{E}[\langle g_V(\theta), \Delta\theta \rangle \mid \theta] \leq -cV(\theta) + \epsilon_{\text{probe}} + C_{\text{fd}}\sigma^2. \quad (65)$$

The expectation is taken over the sampled probe directions. This condition states that the finite-probe CPI direction decreases the spectral curvature violation in expectation, up to probe coverage error and finite-difference error. It is a transfer condition from directional curvature descent to spectral curvature violation; such a condition is unavoidable for finite-probe methods, since finitely many sampled directions cannot unconditionally control the full Hessian spectrum.

**Assumption B.4 (local upper smoothness of the violation objective).** Along the CPI warm-up trajectory,  $V$  admits the local quadratic upper expansion

$$V(\theta + \Delta) \leq V(\theta) + \langle g_V(\theta), \Delta \rangle + \frac{L_V}{2} \|\Delta\|^2, \quad g_V(\theta) \in \partial V(\theta), \quad (66)$$

for sufficiently small  $\Delta$ . In addition, the CPI update is bounded:

$$\|\Delta\theta\| \leq D. \quad (67)$$

**Proposition B.5 (one-step decrease of spectral curvature violation).** Under Assumptions B.3 and B.4, the CPI warm-up step

$$\theta^+ = \theta + \eta\Delta\theta \quad (68)$$

satisfies

$$\mathbb{E}[V(\theta^+) \mid \theta] \leq (1 - \eta c)V(\theta) + \eta\epsilon_{\text{probe}} + \eta C_{\text{fd}}\sigma^2 + \frac{L_V D^2}{2} \eta^2. \quad (69)$$

**Proof.** By Assumption B.4 with  $\Delta = \eta\Delta\theta$ ,

$$V(\theta + \eta\Delta\theta) \leq V(\theta) + \eta\langle g_V(\theta), \Delta\theta \rangle + \frac{L_V}{2} \eta^2 \|\Delta\theta\|^2. \quad (70)$$

Taking conditional expectation yields

$$\mathbb{E}[V(\theta^+) \mid \theta] \leq V(\theta) + \eta\mathbb{E}[\langle g_V(\theta), \Delta\theta \rangle \mid \theta] + \frac{L_V D^2}{2} \eta^2. \quad (71)$$

Using Assumption B.3, we obtain

$$\mathbb{E}[V(\theta^+) \mid \theta] \leq V(\theta) + \eta(-cV(\theta) + \epsilon_{\text{probe}} + C_{\text{fd}}\sigma^2) + \frac{L_V D^2}{2} \eta^2, \quad (72)$$

which gives

$$\mathbb{E}[V(\theta^+) \mid \theta] \leq (1 - \eta c)V(\theta) + \eta\epsilon_{\text{probe}} + \eta C_{\text{fd}}\sigma^2 + \frac{L_V D^2}{2} \eta^2. \quad (73)$$

This proves the claim.

**Corollary B.6 (approach to a relaxed curvature-controlled region).** Suppose the conditions of Proposition B.5 hold. If CPI uses a fixed step size  $\eta \in (0, 1/c)$ , then after  $S$  warm-up steps,

$$\mathbb{E}[V(\theta_S)] \leq (1 - c\eta)^S V(\theta_0) + \frac{\varepsilon_{\text{probe}}}{c} + \frac{C_{\text{fd}}\sigma^2}{c} + \frac{L_V D^2}{2c} \eta. \quad (74)$$

Thus, CPI drives the expected curvature violation to an  $O(\varepsilon_{\text{probe}} + \sigma^2 + \eta)$  neighborhood of zero.

Moreover, for any  $\delta > 0$ , define the relaxed curvature region

$$\mathcal{R}_{m-\delta, L+\delta} = \{\theta : (m - \delta)I \preceq H_t(\theta) \preceq (L + \delta)I, \forall t\}. \quad (75)$$

If  $V(\theta_S) \leq \delta$ , then  $\theta_S \in \mathcal{R}_{m-\delta, L+\delta}$ . In the stochastic case, for any  $\delta > 0$ ,

$$\mathbb{P}(\theta_S \notin \mathcal{R}_{m-\delta, L+\delta}) \leq \frac{\mathbb{E}[V(\theta_S)]}{\delta}. \quad (76)$$

Therefore, when the expected violation bound above is small, CPI reaches the relaxed curvature-controlled region with high probability.

**Proof.** Applying Proposition B.5 recursively gives

$$\mathbb{E}[V(\theta_S)] \leq (1 - c\eta)^S V(\theta_0) + \sum_{i=0}^{S-1} (1 - c\eta)^i \left( \eta \varepsilon_{\text{probe}} + \eta C_{\text{fd}} \sigma^2 + \frac{L_V D^2}{2} \eta^2 \right). \quad (77)$$

Since

$$\sum_{i=0}^{S-1} (1 - c\eta)^i \leq \frac{1}{c\eta}, \quad (78)$$

we obtain

$$\mathbb{E}[V(\theta_S)] \leq (1 - c\eta)^S V(\theta_0) + \frac{\varepsilon_{\text{probe}}}{c} + \frac{C_{\text{fd}}\sigma^2}{c} + \frac{L_V D^2}{2c} \eta. \quad (79)$$

This proves the expected-violation bound.

If  $V(\theta_S) \leq \delta$ , then every nonnegative term in the definition of  $V$  is at most  $\delta$ . Hence, for every task  $t$ ,

$$\lambda_{\min}(H_t(\theta_S)) \geq m - \delta, \quad \lambda_{\max}(H_t(\theta_S)) \leq L + \delta. \quad (80)$$

Therefore,

$$\theta_S \in \mathcal{R}_{m-\delta, L+\delta}. \quad (81)$$

Finally, if  $\theta_S \notin \mathcal{R}_{m-\delta, L+\delta}$ , then at least one task has either  $\lambda_{\min}(H_t(\theta_S)) < m - \delta$  or  $\lambda_{\max}(H_t(\theta_S)) > L + \delta$ , which implies  $V(\theta_S) > \delta$ . Thus, by Markov's inequality,

$$\mathbb{P}(\theta_S \notin \mathcal{R}_{m-\delta, L+\delta}) \leq \mathbb{P}(V(\theta_S) > \delta) \leq \frac{\mathbb{E}[V(\theta_S)]}{\delta}. \quad (82)$$

This completes the proof.

**Discussion.** The result above clarifies how the task-wise projected CPI update can be related to the favorable curvature region motivated by Theorem 3.2 under an explicit probe-to-spectrum transfer condition. The sign-based update directly yields descent for the directional curvature surrogate estimated by CPI. To translate this directional descent into a guarantee on the spectral region  $\mathcal{R}_{m, L}$ , one needs the probe-to-spectrum transfer condition in Assumption B.3, since a finite number of random directions cannot unconditionally control the full Hessian spectrum. Under this condition, CPI reduces the spectral curvature violation up to finite-probe, finite-difference, and finite-step errors, and hence reaches a relaxed curvature-controlled region with high probability when the expected violation is small.

## C Limitations

CPI relies on stochastic directional curvature estimates, which provide a computationally tractable proxy rather than a full characterization of the Hessian spectrum. This approximation is practically unavoidable for modern high-dimensional neural networks, where exact Hessian-spectrum analysis is generally prohibitive. Its effectiveness may depend on the finite-difference scale, the number of curvature probes, and warm-up hyperparameters. Our theoretical analysis is local and should be interpreted as a sufficient-condition motivation rather than a global convergence guarantee for deep nonconvex MTL models. Such a scope is inherent to tractable analyses of initialization effects in modern neural networks, where global guarantees are generally out of reach. Finally, although CPI is evaluated on several standard MTL benchmarks, broader validation on larger-scale architectures and additional task combinations remains an important direction for future work.